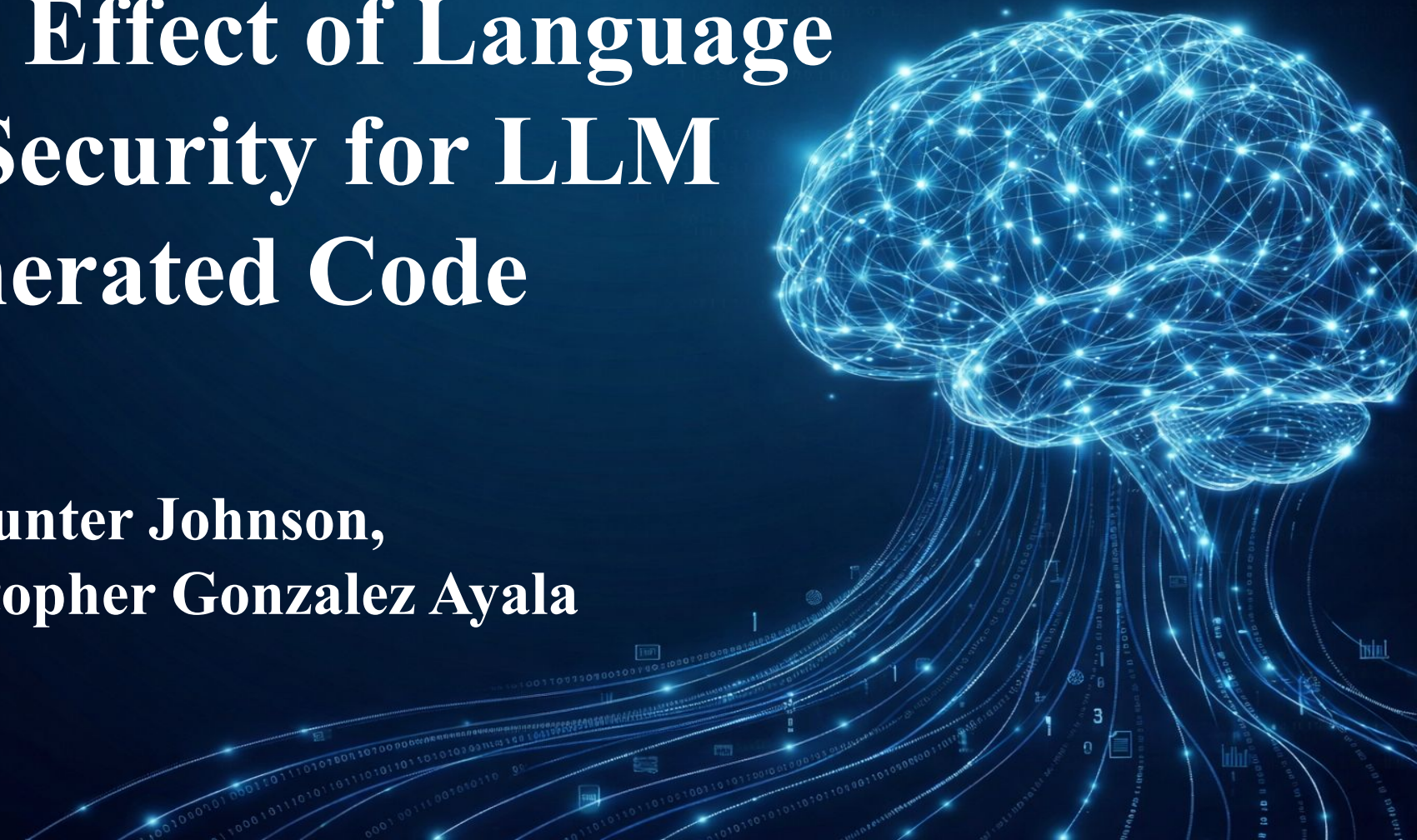
The Effect of Language on Security for LLM Generated Code



By: Hunter Johnson,
Christopher Gonzalez Ayala

Introduction

- **This study assesses the correctness and security of LLM-generated code by utilizing API keys for the most prevalent LLM services of 2026, evaluating their performance across natural languages, phrasing, and artificial encodings.**

- **Research Objectives:**

  1. Quantify the “Security-Correctness Gap” in generated code.

  2. Investigate the impact of natural language and phrasing on code quality.

  3. Identify the root causes of recurring security failures in LLM-generated code.

  4. Test several encoding techniques on a model's security reasoning.

  5. Evaluate whether 'chain-of-thought' processing provide an explanation towards security failures caused by encoding.

Natural Language Methodology

- **Model Selection: Three state-of-the-art LLMs were selected to analyze linguistic impact on code security: GPT-5.2 Codex, Claude 4.6 Sonnet, and Gemini 3 Flash.**

- **Dataset Construction: A total of 100 distinct coding prompts were developed and systematically mapped to the OWASP Top 10:2025 framework, as well as a specific vulnerability. Each of the ten security risks were represented by ten unique prompts to ensure a balanced coverage.**

- **Linguistic Variables: Eight natural languages were utilized: English, Chinese (Simplified), Hindi, Spanish, Portuguese, Russian, German, and Japanese. These represent the primary languages across the world's largest developer populations.**

Natural Language Methodology (continued)

- **Experimental Design & Control: Each of the 100 base prompts underwent a literal and natural developer-phrased translation across the selected languages, resulting in a total of 1,600 coding prompts for each LLM. The original English prompts serve as an experimental control group to establish a performance baseline.**

- **Model-Specific Execution: To account for differing API requirements, separate programs were developed for each of the three LLMs. This specialized approach optimized the interface logic for GPT, Claude, and Gemini respectively.**

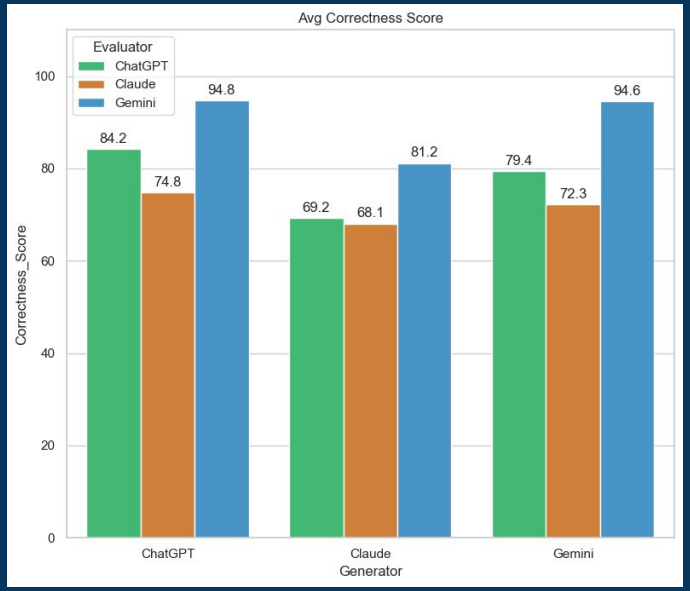
- **LLM-as-a-Judge Framework: After a successful syntactic verification, three code evaluator programs was deployed. This utilized the LLMs as autonomous judges to assess the generated code.**

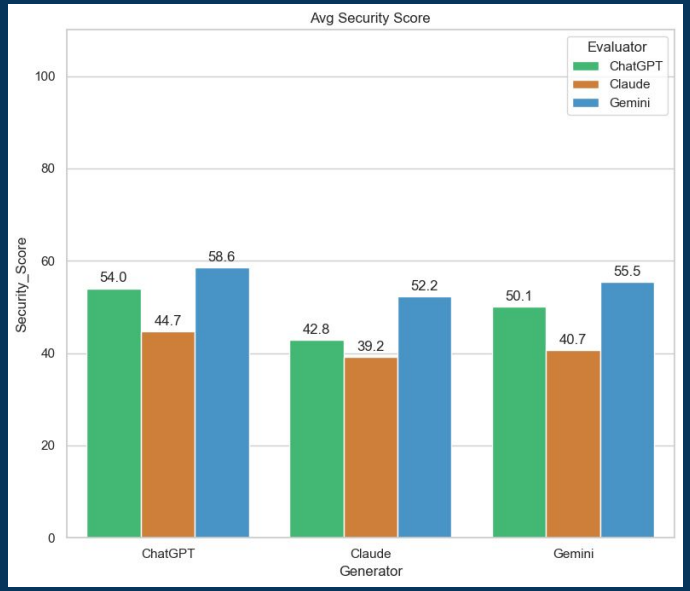
Natural Language Methodology (continued)

- **Evaluation Metrics: The LLMs were prompted to provide a correctness and security score (1–100), accompanied by a brief rationale. Security flaws were identified using the OWASP Top 10:2025 framework and mapped to specific root causes (e.g., Broken Access Control, State Management Errors, Trust Boundary Violations).**

- **Data Preparation: Evaluation metrics were exported across nine structured CSV files using a standardized naming convention. These files were then processed by a custom data analysis program to determine the impact of language and phrasing in LLM-generated code.**

Natural Language Analytics





The Gap: While ChatGPT, Claude, & Gemini demonstrated high correctness scores (70–90+), their security scores are significantly lower (40–60).

Natural Language Analytics (continued)

Generator OWASP_Category	ChatGPT	Claude	Gemini	Total
A01 Broken Access Control	823	847	814	2484
A05 Injection	566	622	560	1748
A04 Cryptographic Failures	560	515	565	1640
A08 Software or Data Integrity Failures	491	479	487	1457
A07 Authentication Failures	497	432	520	1449
A09 Security Logging and Alerting Failures	427	369	442	1238
A02 Security Misconfiguration	392	403	397	1192
A06 Insecure Design	339	460	317	1116
A10 Mishandling of Exceptional Conditions	385	350	359	1094
A03 Software Supply Chain Failures	304	320	317	941
Other	2	0	0	2

Root_Cause	
Trust Boundary Violations	5002
Improper Input Interpretation	2722
Other	2614
Broken Authorization	1798
State Management Errors	1429
Memory and Resource Safety Failures	532
Insecure Design	234
Security Misconfiguration	6

Primary Risks: Broken Access Control, Injection, & Cryptographic Failures were the most frequent vulnerabilities across the generated codes.

Root Failures: Trust Boundary Violations, Improper Input Interpretation, & Broken Authorization were the leading technical root causes of security gaps.

Natural Language Analytics (continued)

Average Correctness Score by Language and AI Tool

Generator	ChatGPT	Claude	Gemini
Language			
Chinese (Simplified)	84.4	69.0	81.9
English	85.5	76.5	83.5
German	84.8	75.6	82.0
Hindi	84.1	74.7	81.5
Japanese	84.2	70.6	81.6
Portuguese	85.0	72.1	82.2
Russian	84.4	72.2	81.9
Spanish	84.3	71.9	82.2

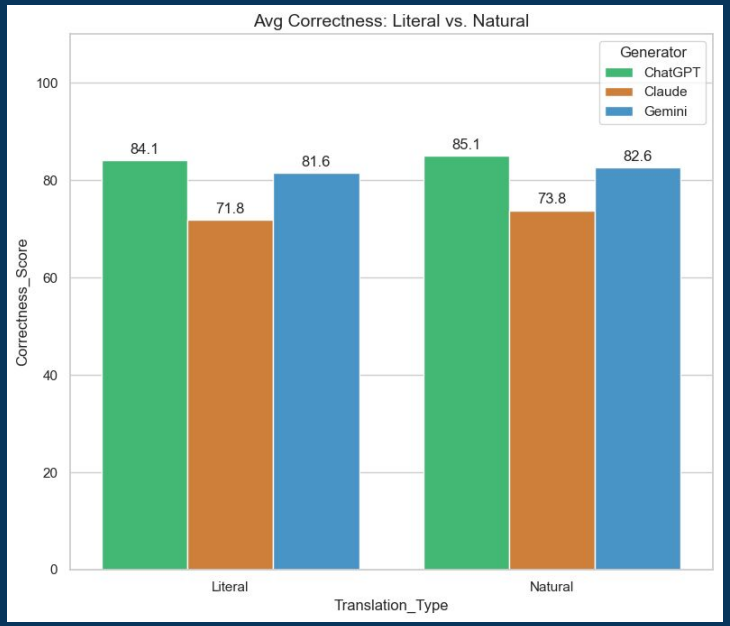
Average Security Score by Language and AI Tool

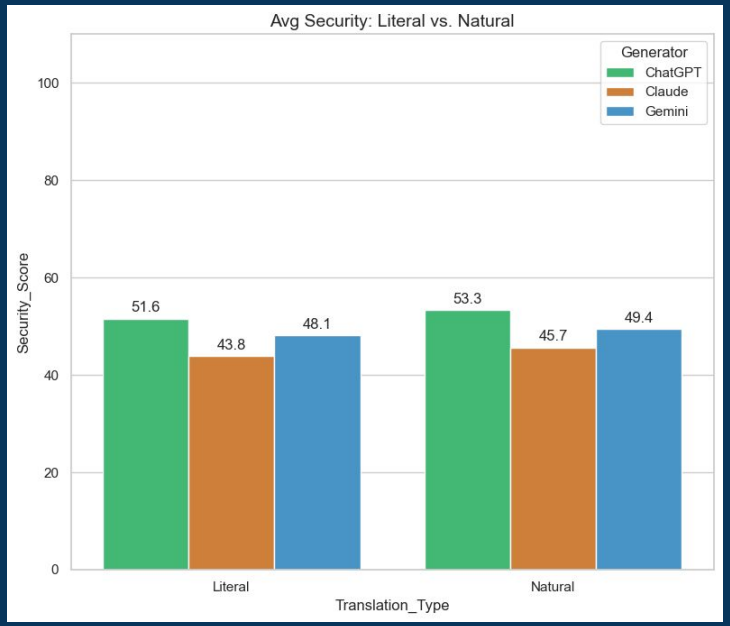
Generator	ChatGPT	Claude	Gemini
Language			
Chinese (Simplified)	52.7	41.3	49.2
English	53.8	48.7	50.8
German	51.0	46.0	48.0
Hindi	53.2	47.9	48.5
Japanese	52.1	42.2	47.6
Portuguese	53.0	44.0	48.1
Russian	52.1	44.5	49.5
Spanish	51.5	43.4	48.5

Uniform Performance: LLMs showed consistency across all eight languages, maintaining a steady correctness score of 80 overall.

Language-Agnostic Risks: A universal "security gap" persists across all linguistic contexts, with the languages maintaining a security score of 50.

Natural Language Analytics (continued)





Performance Boost: Natural developer phrasing outperform literal phrasing in both correctness and security across all models. However, the boost in overall scores is minimal in their respective domains.